

RAPPORT

Programmation Système — Projet Al Massar — Déploiement d'une
Infrastructure Réseau d'Entreprise
(Approche Physique & Approche Virtuelle sous Linux)
3^e année Cybersécurité — École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiants :

ACHOUKHANE Achraf

ATOURABI Hibat Allah

CHEMAOU Aya

EL OUDRI Sara

HALLAL Yahya

HATHOUTI Mohammed Taha

JIDAL Ilyas

KABORE Mohammad Sharif Jonathan

Filière :

Cybersécurité

Année :

2025/2026

Enseignants :

M. Bakhouya

Date :

3 mai 2026

Remerciements

Ce projet a été réalisé dans le cadre du cours d'Administration Réseau dispensé par **M. Bakhouya, M. Noufel, M. Aqachtoul, M. ElAmrani & M. Staifi** à l'UIR durant l'année universitaire 2025-2026.

Je tiens à remercier les enseignants pour la qualité de leur encadrement, et en particulier pour nous avoir proposé un projet ambitieux permettant d'appréhender la mise en place d'une infrastructure réseau d'entreprise complète, tant en environnement physique (équipements Cisco et HP réels) qu'en environnement virtualisé (Linux Network Namespaces).

Nous tenons également à adresser nos sincères remerciements à **M. QAMRI Abdelhadi & M. BERDAY Mourad** qui nous ont chaleureusement accueillis dans leur bureau, lequel a servi de laboratoire tout au long de ce projet. Leur disponibilité, leur patience et le temps qu'ils ont consacré à nous transmettre des connaissances plus approfondies et à nous expliquer des aspects techniques avancés ont été d'une aide précieuse.

Je souhaite également remercier les enseignants des formations antérieures :

- Les professeurs de l'**Université Grenoble Alpes** (INF301, INF304, INF401) pour les bases solides en systèmes ;
- Les professeurs de **Polytech Grenoble** (PROG C, CPS, ALG, ALM, Réseau) pour la maîtrise de Linux et de la programmation système qui a facilité la partie virtualisation ainsi qu'en manipulation de matériel réseau.

Achraf ACHOUKHANE

Hibat Allah ATOURABI

Aya CHEMAOU

Sara EL OUDRI

Yahya HALLAL

Mohammed Taha HATHOUTI

Ilyas JIDAL

Mohammad Sharif Jonathan KABORE

Avril 2026

Table des matières

1	Introduction et Contexte du Projet	4
2	Architecture Globale du Réseau Al Massar	5
2.1	Topologie logique	5
2.2	Plan d'adressage	5
3	Partie I — Déploiement Physique	6
3.1	Routeur Cisco 2900	6
3.1.1	Configuration de base et interfaces	6
3.1.2	Route statique par défaut	6
3.1.3	NAT Overload (PAT)	7
3.2	Switch Cœur — Catalyst 3650 (L3)	8
3.2.1	Configuration de base	8
3.2.2	Création des VLANs	8
3.2.3	Interfaces SVI (passerelles inter-VLAN)	9
3.2.4	Route par défaut et sauvegarde	9
3.3	Switches d'Accès HP 2920	10
3.3.1	HP 2920 #1 — VLANs 10 et 20	10
3.3.2	HP 2920 #2 — VLANs 30 et 40	11
3.4	Serveur Ubuntu — DHCP	13
3.4.1	Installation et configuration réseau du serveur	13
3.4.2	Configuration des pools DHCP	14
3.5	Serveur Ubuntu — DNS (BIND9)	14
3.5.1	Configuration de BIND9	15
3.6	Serveur Ubuntu — HTTPS (Nginx)	16
3.7	Serveur Ubuntu — FTPS (vsftpd)	18
3.8	Tests de connectivité physique	19
3.8.1	Vérification inter-VLAN	19
3.8.2	Vérification DNS	20

4	Partie II — Déploiement Virtuel (Linux Network Namespaces)	21
4.1	Concept et Architecture	21
4.1.1	Namespaces créés	21
4.1.2	Plan d’adressage virtuel	21
4.2	Interface TUI — CONF_WINDOW.sh	21
4.3	Orchestrateur Principal — BOOT.sh	23
4.4	Infrastructure L2 — Bridges et Câblage	25
4.4.1	CREATE_L2_SWITCHES.sh — Création des bridges	25
4.4.2	PCs.sh — Câblage des postes clients	25
4.4.3	UBUNTU_SERVER.sh — Câblage du serveur Ubuntu	26
4.5	Logique VLAN — CREATE_VLANS.sh	26
4.6	Transit L3 et NAT — CONNECT_CORE_TO_ROUTER.sh & SETUP_NAT.sh	27
4.6.1	Lien transit sw-core ↔ router	27
4.6.2	NAT et accès Internet	27
4.7	Services du Serveur Ubuntu Virtuel	28
4.7.1	DHCP — SETUP_DHCP.sh	28
4.7.2	DNS — SETUP_DNS.sh	29
4.7.3	HTTPS — SETUP_HTTPS.sh	30
4.7.4	FTPS — SETUP_FTPS.sh	31
4.8	ACL et Segmentation — ACL.sh	32
4.9	Tests et Validation — ALL_TESTS.sh	33
5	Comparaison des Deux Approches	35
	Conclusion	35
	Références	36

1 Introduction et Contexte du Projet

L'entreprise fictive **Al Massar** (« La Voie ») est une société de conseil et de recrutement souhaitant déployer une infrastructure réseau segmentée, sécurisée et fournissant des services internes (DHCP, DNS, Web sécurisé, FTP sécurisé) accessibles par l'ensemble de ses départements, tout en permettant un accès Internet supervisé.

Ce projet a été réalisé **deux fois**, selon deux approches radicalement différentes afin d'en comparer la mise en œuvre, les contraintes et les résultats :

- **Partie I — Infrastructure physique** : équipements réels d'un laboratoire réseau (routeur Cisco 2900, switch cœur Catalyst 3650, switches d'accès HP 2920, serveur Ubuntu physique) ;
- **Partie II — Infrastructure virtuelle** : émulation complète sur une seule machine Linux grâce aux *Linux Network Namespaces*, aux bridges et aux interfaces *veth*, sans aucune machine virtuelle lourde.

Les deux approches reproduisent la même architecture logique : segmentation en VLANs, routage inter-VLAN, NAT pour l'accès WAN, et un serveur Ubuntu hébergeant les services DHCP, DNS, HTTPS et FTPS.

2 Architecture Globale du Réseau Al Massar

2.1 Topologie logique

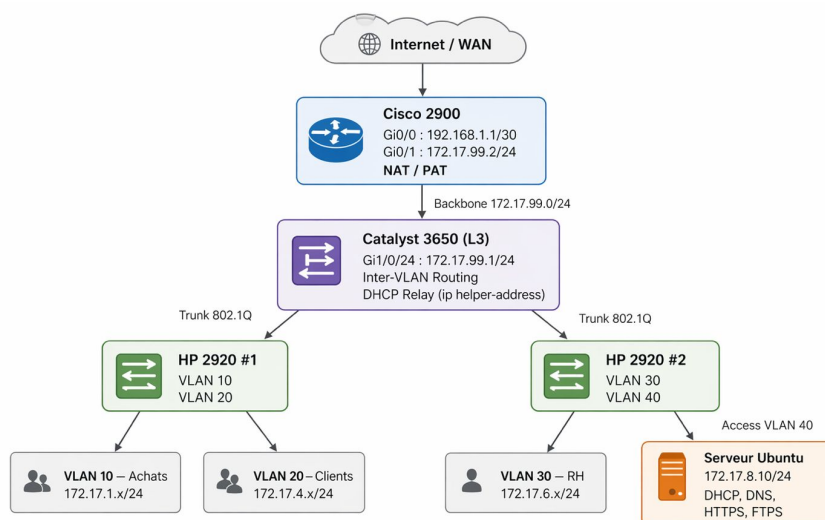


FIGURE 1 – Topologie physique du réseau Al Massar

La topologie repose sur trois niveaux :

1. **Couche WAN / Bordure** : le routeur Cisco 2900 assure la connectivité Internet via NAT overload ;
2. **Couche cœur (L3)** : le Catalyst 3650 réalise le routage inter-VLAN (SVI), relaie les requêtes DHCP (`ip helper-address`) et se connecte au routeur via une interface routée (`no switchport`) ;
3. **Couche accès (L2)** : deux switches HP 2920 distribuent les VLANs aux postes clients.

2.2 Plan d'adressage

VLAN	Département	Réseau	Passerelle (SVI)	Switch
10	Purchases (Achats)	172.17.1.0/24	172.17.1.1	HP #1
20	Clients	172.17.4.0/24	172.17.4.1	HP #1
30	HR (Ressources H.)	172.17.6.0/24	172.17.6.1	HP #2
40	Services Ubuntu	172.17.8.0/24	172.17.8.1	HP #2

Équipement	Rôle	Adresse(s) clé(s)
Cisco 2900	Routeur bordure / NAT	Gi0/0 : 192.168.1.1 (WAN) Gi0/1 : 172.17.99.2
Catalyst 3650	Switch L3 / Routage inter-VLAN	Interface routée uplink : 172.17.99.1/24
HP 2920 #1	Switch accès VLANs 10 & 20	—
HP 2920 #2	Switch accès VLANs 30 & 40	—
Ubuntu Server	DHCP + DNS + HTTPS + FTPS	172.17.8.10/24

3 Partie I — Déploiement Physique

3.1 Routeur Cisco 2900

3.1.1 Configuration de base et interfaces

Le routeur porte le nom d'hôte **ROUTER**. Deux interfaces GigabitEthernet sont utilisées :

- **Gi0/0** : interface WAN, IP fixe 192.168.1.1 (côté ISP/Internet) ;
- **Gi0/1** : interface LAN, IP 172.17.99.2/24 connectée directement à l'interface routée du Catalyst 3650 (172.17.99.1/24).

```
ROUTER#show ip interface brief
Any interface listed with OK? value "NO" does not have a valid configuration

Interface                IP-Address      OK? Method Status        Protocol
Embedded-Service-Engine0/0 unassigned      YES unset  administratively down  down
GigabitEthernet0/0        192.168.1.1     YES manual  down          down
GigabitEthernet0/1        172.17.99.2     YES manual  down          down
NVI0                      unassigned      NO  unset  up            up
```

FIGURE 2 – État des interfaces du routeur Cisco — `show ip interface brief`

```
ROUTER(config)#interface GigabitEthernet0/1
ROUTER(config-if)#description Uplink_to_3650
ROUTER(config-if)#ip address 172.17.99.2 255.255.255.0
ROUTER(config-if)#ip nat inside
ROUTER(config-if)#no shutdown
ROUTER(config-if)#exit
```

FIGURE 3 – Configuration de l'interface GigabitEthernet 0/1 (172.17.99.2/24)

3.1.2 Route statique par défaut

La route par défaut est configurée manuellement, pointant vers le next-hop de l'ISP (192.168.1.2) :

```
ROUTER(config)#ip route 0.0.0.0 0.0.0.0 192.168.1.2
ROUTER(config)#
```

FIGURE 4 – Configuration de la route statique par défaut (0.0.0.0/0 via 192.168.1.2)

Listing 1 – Route statique par défaut — Cisco IOS

```
1 ROUTER(config)#ip route 0.0.0.0 0.0.0.0 192.168.1.2
```

3.1.3 NAT Overload (PAT)

L'ACL standard NAT_NETWORKS identifie les sous-réseaux internes :

```
ROUTER(config)#ip access-list standard NAT_NETWORKS
ROUTER(config-std-nacl)#permit 172.17.1.0 0.0.0.255
ROUTER(config-std-nacl)#permit 172.17.4.0 0.0.0.255
ROUTER(config-std-nacl)#permit 172.17.6.0 0.0.0.255
ROUTER(config-std-nacl)#permit 172.17.8.0 0.0.0.255
ROUTER(config-std-nacl)#permit 172.17.99.0 0.0.0.255
ROUTER(config-std-nacl)#exit
ROUTER(config)#
```

FIGURE 5 – Définition de l'ACL standard NAT_NETWORKS (permis de tous les VLANs)

Listing 2 – ACL NAT et configuration NAT overload

```
1 ROUTER(config)#ip access-list standard NAT_NETWORKS
2 ROUTER(config-std-nacl)#permit 172.17.1.0 0.0.0.255
3 ROUTER(config-std-nacl)#permit 172.17.4.0 0.0.0.255
4 ROUTER(config-std-nacl)#permit 172.17.6.0 0.0.0.255
5 ROUTER(config-std-nacl)#permit 172.17.8.0 0.0.0.255
6 ROUTER(config-std-nacl)#permit 172.17.99.0 0.0.0.255
7 ROUTER(config-std-nacl)#exit
8 ROUTER(config)#ip nat inside source list NAT_NETWORKS
9 interface GigabitEthernet0/0 overload
```

```
ROUTER(config)#$ list NAT_NETWORKS interface GigabitEthernet0/0 overload
ROUTER(config)#end
ROUTER#
Apr 10 15:46:45.847: %SYS-5-CONFIG_I: Configured from console by console
ROUTER#
ROUTER#wr
Building configuration...
[OK]
ROUTER#
```

FIGURE 6 – Application du NAT overload sur l'interface WAN Gi0/0

Le NAT overload (*PAT*) traduit les adresses privées des cinq VLANs en une unique adresse publique sur Gi0/0, en différenciant les flux par les numéros de port source. La table de traduction active confirme que des sessions sont bien en cours :

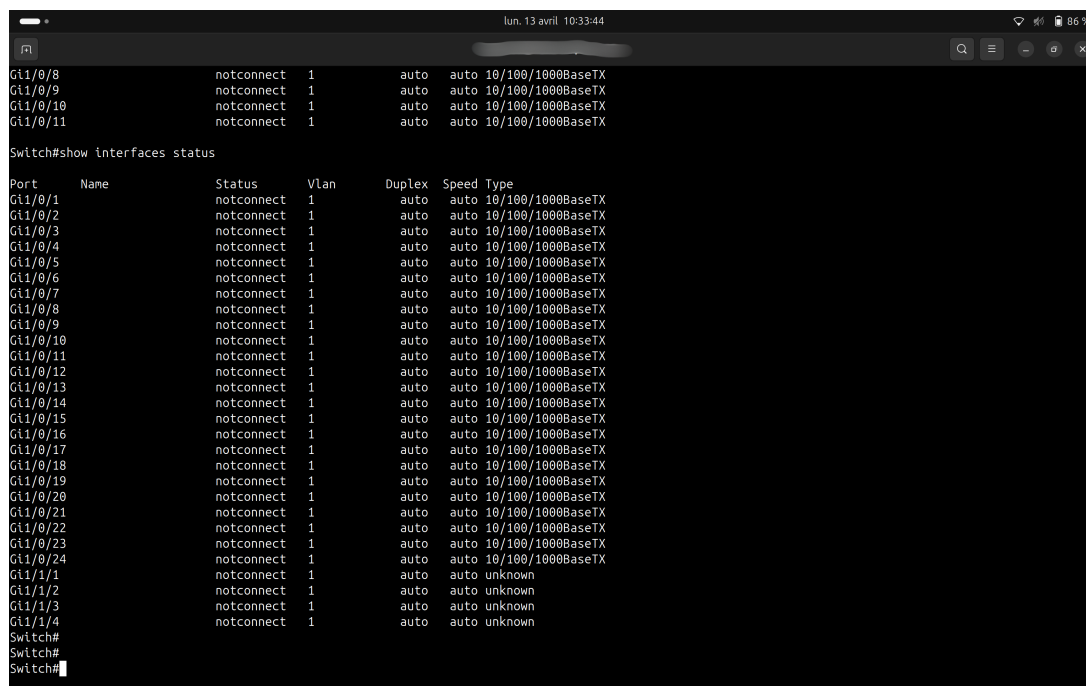
```
Router#show ip nat translations
Router#show running-config | section nat
ip nat outside
ip nat inside
ip nat inside source list 10 interface GigabitEthernet0/0 overload
Router#show access-lists 10
Standard IP access list 10
10 permit 172.17.0.0, wildcard bits 0.0.255.255
```

FIGURE 7 – Vérification des traductions NAT actives (show ip nat translations)

3.2 Switch Cœur — Catalyst 3650 (L3)

Le Catalyst 3650 joue le rôle de **switch de distribution / cœur de réseau L3** : il crée les VLANs, porte les SVI (Switch Virtual Interfaces) servant de passerelles inter-VLAN, et relaie les requêtes DHCP via `ip helper-address`.

3.2.1 Configuration de base



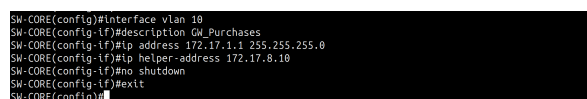
```
Switch#show interfaces status
Port      Name      Status    Vlan    Duplex  Speed  Type
G11/0/1    notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/2    notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/3    notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/4    notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/5    notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/6    notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/7    notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/8    notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/9    notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/10   notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/11   notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/12   notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/13   notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/14   notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/15   notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/16   notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/17   notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/18   notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/19   notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/20   notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/21   notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/22   notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/23   notconnect 1         auto    auto    10/100/1000BaseTX
G11/0/24   notconnect 1         auto    auto    10/100/1000BaseTX
G11/1/1    notconnect 1         auto    auto    unknown
G11/1/2    notconnect 1         auto    auto    unknown
G11/1/3    notconnect 1         auto    auto    unknown
G11/1/4    notconnect 1         auto    auto    unknown
Switch#
Switch#
Switch#
```

FIGURE 8 – Configuration initiale du Catalyst 3650 (hostname, ip routing)

La commande `ip routing` est indispensable : sans elle, le Catalyst opère exclusivement en mode L2 et est incapable d’acheminer les paquets entre VLANs. Le nom d’hôte (Switch) est personnalisé pour faciliter l’identification en cas d’intervention.

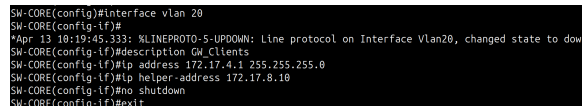
3.2.2 Création des VLANs

Cinq VLANs sont créés sur le Catalyst : 10 (Purchases), 20 (Clients), 30 (HR), 40 (Services) et 99 (Backbone).



```
SW-CORE(config)#interface vlan 10
SW-CORE(config-if)#description GW_Purchases
SW-CORE(config-if)#ip address 172.17.1.1 255.255.255.0
SW-CORE(config-if)#ip helper-address 172.17.8.10
SW-CORE(config-if)#no shutdown
SW-CORE(config-if)#exit
SW-CORE(config)#
```

FIGURE 9 – Création VLAN 10



```
SW-CORE(config)#interface vlan 20
SW-CORE(config-if)#
*Apr 13 10:19:45.333: %LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan20, changed state to down
SW-CORE(config-if)#description GW_Clients
SW-CORE(config-if)#ip address 172.17.4.1 255.255.255.0
SW-CORE(config-if)#ip helper-address 172.17.8.10
SW-CORE(config-if)#no shutdown
SW-CORE(config-if)#exit
```

FIGURE 10 – Création VLAN 20

Les VLANs 10 (*Purchases*) et 20 (*Clients*) sont associés au premier switch d’accès HP #1. Les VLANs 30 (*HR*) et 40 (*Services*) sont quant à eux distribués par HP #2, qui héberge également la machine Ubuntu :

```
SW-CORE(config)#interface vlan 30
SW-CORE(config-if)#
*Apr 13 10:23:44.525: %LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan30, changed state to down
SW-CORE(config-if)#description GW_HR
SW-CORE(config-if)#ip address 172.17.6.1 255.255.255.0
SW-CORE(config-if)#ip helper-address 172.17.8.10
SW-CORE(config-if)#no shutdown
SW-CORE(config-if)#exit
```

FIGURE 11 – Création VLAN 30

```
SW-CORE(config)#interface vlan 40
SW-CORE(config-if)#
*Apr 13 10:29:06.523: %LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan40, changed state to down
SW-CORE(config-if)#description GW_Services
SW-CORE(config-if)#ip address 172.17.8.1 255.255.255.0
SW-CORE(config-if)#no shutdown
SW-CORE(config-if)#exit
```

FIGURE 12 – Création VLAN 40

3.2.3 Interfaces SVI (passerelles inter-VLAN)

Chaque VLAN reçoit une interface virtuelle SVI portant l'adresse de passerelle. Le VLAN 40 (serveur) est configuré avec `ip helper-address` pour relayer les requêtes DHCP des autres VLANs vers le serveur Ubuntu.

```
SW-CORE(config)#interface GigabitEthernet1/0/22
SW-CORE(config-if)#description Trunk_to_HP1
SW-CORE(config-if)#switchport mode trunk
SW-CORE(config-if)#switchport trunk allowed vlan 10,20
SW-CORE(config-if)#no shutdown
SW-CORE(config-if)#exit
```

FIGURE 13 – SVI VLAN 10 (172.17.1.1) et VLAN 20 (172.17.4.1)

```
SW-CORE(config)#interface GigabitEthernet1/0/23
SW-CORE(config-if)#description Trunk_to_HP2
SW-CORE(config-if)#switchport mode trunk
SW-CORE(config-if)#switchport trunk allowed vlan 30,40
SW-CORE(config-if)#no shutdown
SW-CORE(config-if)#exit
```

FIGURE 14 – SVI VLAN 30 (172.17.6.1) et VLAN 40 (172.17.8.1) + helper

La directive `ip helper-address 172.17.8.10` sur la SVI VLAN 40 est le mécanisme clé du relaying DHCP : elle transforme le broadcast *DHCP Discover* émis par un client en une requête unicast à destination du serveur Ubuntu. Sans elle, les requêtes DHCP des clients s'arrêteraient au premier switch d'accès. L'interface uplink vers le routeur complète le câblage logique : au lieu d'utiliser un VLAN 99 dédié, le port physique reliant le Catalyst au routeur est configuré en **interface routée** (`no switchport`) avec l'adresse 172.17.99.1/24, ce qui est plus direct et évite la création d'un VLAN supplémentaire.

```
SW-CORE(config)#interface GigabitEthernet1/0/24
SW-CORE(config-if)#description Uplink_to_R1
SW-CORE(config-if)#switchport mode trunk
SW-CORE(config-if)#switchport trunk allowed vlan 99
SW-CORE(config-if)#no shutdown
SW-CORE(config-if)#exit
```

FIGURE 15 – Interface routée uplink vers le routeur (`no switchport`, 172.17.99.1/24)

3.2.4 Route par défaut et sauvegarde

```
SW-CORE(config)#ip route 0.0.0.0 0.0.0.0 172.17.99.2
```

FIGURE 16 – Route par défaut du Catalyst 3650 (via 172.17.99.2 — routeur)

```
SW-CORE#write memory
Building configuration...
Compressed configuration from 3307 bytes to 1714 bytes[OK]
```

FIGURE 17 – Sauvegarde de la configuration (`write memory`)

```

Switch(config)#hostname SW-CORE
SW-CORE(config)#no ip domain-lookup
SW-CORE(config)#enable secret
SW-CORE(config)#ip routing
SW-CORE(config)#vlan 10
SW-CORE(config-vlan)#name Purchases
SW-CORE(config-vlan)#exit
SW-CORE(config)#vlan 20
SW-CORE(config-vlan)#name Clients
SW-CORE(config-vlan)#exit
SW-CORE(config)#vlan 30
SW-CORE(config-vlan)#name HR
SW-CORE(config-vlan)#exit
SW-CORE(config)#vlan 40
SW-CORE(config-vlan)#name Services
SW-CORE(config-vlan)#exit
SW-CORE(config)#vlan 99
SW-CORE(config-vlan)#name Backbone
SW-CORE(config-vlan)#exit
SW-CORE(config)#

```

FIGURE 18 – Vue d’ensemble des VLANs configurés sur le Catalyst 3650

Le Catalyst est maintenant entièrement configuré : cinq VLANs créés, cinq SVIs actives, route par défaut vers le routeur, et configuration sauvegardée en mémoire non volatile (**write memory**). Les switches d’accès HP 2920 peuvent à présent être configurés pour distribuer ces VLANs jusqu’aux postes clients.

3.3 Switches d’Accès HP 2920

3.3.1 HP 2920 #1 — VLANs 10 et 20

Le premier switch HP est connecté aux postes du département Achats (VLAN 10) et du département Clients (VLAN 20). Son port uplink vers le Catalyst est configuré en **trunk** (tagged 802.1Q).

```

HP-2920-24G(config)# show vlans

Status and Counters - VLAN Information

Maximum VLANs to support : 256
Primary VLAN : DEFAULT_VLAN
Management VLAN :

VLAN ID Name | Status | Voice | Jumbo
-----+-----+-----+-----
1    DEFAULT_VLAN | Port-based | No | No
10   Purchases   | Port-based | No | No
20   Clients     | Port-based | No | No

HP-2920-24G(config)#

```

FIGURE 19 – État des VLANs sur HP #1 (**show vlans**) — VLAN 10 Purchases, VLAN 20 Clients

```
HP-2920-24G(config)# vlan 10
HP-2920-24G(vlan-10)# name "Purchases"
HP-2920-24G(vlan-10)# untagged 1-10
HP-2920-24G(vlan-10)# tagged 24
HP-2920-24G(vlan-10)# exit
HP-2920-24G(config)#
```

FIGURE 20 – Création et nommage VLAN 10 sur HP #1

```
HP-2920-24G(config)# vlan 20
HP-2920-24G(vlan-20)# name "Clients"
HP-2920-24G(vlan-20)# untagged 11-20
HP-2920-24G(vlan-20)# tagged 24
HP-2920-24G(vlan-20)# exit
HP-2920-24G(config)#
```

FIGURE 21 – Création et nommage VLAN 20 sur HP #1

```
SW-ACCESS-1(config)# vlan 10
SW-ACCESS-1(vlan-10)# untagged 1-12
SW-ACCESS-1(vlan-10)# exit
SW-ACCESS-1(config)# vlan 20
SW-ACCESS-1(vlan-20)# untagged 13-23
SW-ACCESS-1(vlan-20)# exit
SW-ACCESS-1(config)# write memory
SW-ACCESS-1(config)#
```

FIGURE 22 – Attribution de la plage de ports aux VLANs (HP #1)

```
HP-2920-24G(config)# vlan 1
HP-2920-24G(vlan-1)# no untagged 24
HP-2920-24G(vlan-1)# exit
HP-2920-24G(config)# exit
```

FIGURE 23 – Retrait du port 24 (uplink) du VLAN 1 natif

Le port 24, utilisé comme liaison trunk vers le Catalyst (tagged 802.1Q), doit être retiré du VLAN 1 natif : laisser un port trunk dans le VLAN 1 peut provoquer des boucles L2 ou des fuites de trafic. Une fois retiré, on vérifie que les ports d'accès sont bien affectés en mode *untagged* à leurs VLANs respectifs :

VLAN ID : 10				
Name : Purchases				
Status : Port-based				
Voice : No				
Jumbo : No				
Port	Information	Mode	Unknown VLAN	Status
1	Untagged	Learn	Down	
2	Untagged	Learn	Down	
3	Untagged	Learn	Down	
4	Untagged	Learn	Down	
5	Untagged	Learn	Down	
6	Untagged	Learn	Down	
7	Untagged	Learn	Down	
8	Untagged	Learn	Down	
9	Untagged	Learn	Down	
10	Untagged	Learn	Down	
11	Untagged	Learn	Down	
12	Untagged	Learn	Down	
24	Tagged	Learn	Up	

FIGURE 24 – Vérification ports VLAN 10 (untagged)

VLAN ID : 20				
Name : Clients				
Status : Port-based				
Voice : No				
Jumbo : No				
Port	Information	Mode	Unknown VLAN	Status
13	Untagged	Learn	Down	
14	Untagged	Learn	Down	
15	Untagged	Learn	Down	
16	Untagged	Learn	Down	
17	Untagged	Learn	Down	
18	Untagged	Learn	Down	
19	Untagged	Learn	Down	
20	Untagged	Learn	Down	
21	Untagged	Learn	Down	
22	Untagged	Learn	Down	
23	Untagged	Learn	Down	
24	Tagged	Learn	Up	

FIGURE 25 – Vérification ports VLAN 20 (untagged)

3.3.2 HP 2920 #2 — VLANs 30 et 40

Le deuxième switch HP dessert le département RH (VLAN 30) et le réseau des serveurs (VLAN 40, où est branché l'Ubuntu Server).


```
SW-ACCESS-2(config)# show vlans

Status and Counters - VLAN Information

Maximum VLANs to support : 256
Primary VLAN : DEFAULT_VLAN
Management VLAN :
```

VLAN ID	Name	Status	Voice	Jumbo
1	DEFAULT_VLAN	Port-based	No	No
30	HR	Port-based	No	No
40	Services	Port-based	No	No

FIGURE 26 – État des VLANs sur HP #2 (show vlans) — VLAN 30 HR, VLAN 40 Services

```
SW-ACCESS-2(config)# vlan 40
SW-ACCESS-2(vlan-40)# name Services
SW-ACCESS-2(vlan-40)# untagged 11
SW-ACCESS-2(vlan-40)# tagged 24
SW-ACCESS-2(vlan-40)# exit
```

FIGURE 27 – Création du VLAN 40 sur HP #2

```
SW-ACCESS-2(config)# show vlans

Status and Counters - VLAN Information

Maximum VLANs to support : 256
Primary VLAN : DEFAULT_VLAN
Management VLAN :
```

VLAN ID	Name	Status	Voice	Jumbo
1	DEFAULT_VLAN	Port-based	No	No
30	HR	Port-based	No	No
40	Services	Port-based	No	No

FIGURE 28 – Retrait du port 24 (uplink) du VLAN 1 sur HP #2

La même procédure est appliquée sur HP #2. Les ports d'accès sont répartis entre VLAN 30 (postes RH) et VLAN 40 (port dédié au serveur Ubuntu). La vérification confirme l'affectation correcte :

```
SW-ACCESS-1(config)# vlan 10
SW-ACCESS-1(vlan-10)# untagged 1-12
SW-ACCESS-1(vlan-10)# exit
SW-ACCESS-1(config)# vlan 20
SW-ACCESS-1(vlan-20)# untagged 13-23
SW-ACCESS-1(vlan-20)# exit
SW-ACCESS-1(config)# write memory
SW-ACCESS-1(config)#
```

FIGURE 29 – Attribution des plages de ports aux VLANs (HP #2)

```
VLAN ID : 30
Name : HR
Status : Port-based
Voice : No
Jumbo : No
```

Port	Information	Mode	Unknown VLAN	Status
1	Untagged	Learn	Down	Down
2	Untagged	Learn	Down	Down
3	Untagged	Learn	Down	Down
4	Untagged	Learn	Down	Down
5	Untagged	Learn	Down	Down
6	Untagged	Learn	Down	Down
7	Untagged	Learn	Down	Down
8	Untagged	Learn	Down	Down
9	Untagged	Learn	Down	Down
10	Untagged	Learn	Down	Down
11	Untagged	Learn	Down	Down
12	Untagged	Learn	Down	Down
24	Tagged	Learn	Up	Up

FIGURE 30 – Vérification ports VLAN 30 (untagged)

3.4 Serveur Ubuntu — DHCP

Le serveur DHCP (`isc-dhcp-server`) est installé sur la machine Ubuntu (IP fixe 172.17.8.10/24), elle-même en VLAN 40. Le Catalyst 3650 relaie les requêtes DHCP des autres VLANs via `ip helper-address 172.17.8.10` configuré sur chaque SVI client.

3.4.1 Installation et configuration réseau du serveur

```
root@ubuntu:~# sudo apt update;
root@ubuntu:~# sudo apt install isc-dhcp-server -y
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:3 https://brave-browser-apt-release.s3.brave.com stable InRelease
Hit:4 https://packages.microsoft.com/repos/code stable InRelease
Hit:5 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:6 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
29 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Suggested packages:
  isc-dhcp-server-ltdm policycoreutils
The following NEW packages will be installed:
  isc-dhcp-server
0 upgraded, 1 newly installed, 0 to remove and 29 not upgraded.
Need to get 1,236 kB of archives.
After this operation, 4,151 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 isc-dhcp-server amd64 4.4.3-P1-4ubuntu2 [1,236 kB]
Fetched 1,236 kB in 4s (348 kB/s)
Preconfiguring packages ...
Selecting previously unselected package isc-dhcp-server.
(Reading database ... 217228 files and directories currently installed.)
Preparing to unpack .../isc-dhcp-server_4.4.3-P1-4ubuntu2_amd64.deb ...
Unpacking isc-dhcp-server (4.4.3-P1-4ubuntu2) ...
Setting up isc-dhcp-server (4.4.3-P1-4ubuntu2) ...
Generating /etc/default/isc-dhcp-server ...
Created symlink /etc/systemd/system/multi-user.target.wants/isc-dhcp-server.service → /usr/lib/systemd/system/isc-dhcp-server.service.
Created symlink /etc/systemd/system/multi-user.target.wants/isc-dhcp-server6.service → /usr/lib/systemd/system/isc-dhcp-server6.service.
Processing triggers for man-db (2.12.0-4build2) ...
```

FIGURE 31 – Installation du paquet `isc-dhcp-server`

```
root@ubuntu:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp108s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
    link/ether 08:00:27:00:00:00 brd ff:ff:ff:ff:ff:ff
    inet 172.17.8.10/24 scope global dynamic noprefixroute
        valid_lft 2410930sec preferred_lft 2410930sec
    inet6 fe80::208:10ff:fe00:0000/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
root@ubuntu:~# cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.4 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.4 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo
```

FIGURE 32 – Console Ubuntu Server (accès initial)

```
GNU nano 7.2
network:
version: 2
ethernets:
  enp108s0:
    dhcp4: false
    addresses:
      - 172.17.8.10/24
    routes:
      - to: default
        via: 172.17.8.1
    nameservers:
      addresses:
        - 127.0.0.1
```

FIGURE 33 – Fichier Netplan YAML — IP statique sur `enp108s0`

```
root@ubuntu:~# sudo netplan apply
(generate:18224): WARNING **: 11:36:16.431: Permissions for /etc/netplan/01-netcfg.yaml are too open. Netplan configuration should NOT be accessible by others.
(generate:18224): WARNING **: 11:36:16.431: Permissions for /etc/netplan/01-network-manager-all.yaml are too open. Netplan configuration should NOT be accessible by others.
(process:18222): WARNING **: 11:36:16.570: Permissions for /etc/netplan/01-netcfg.yaml are too open. Netplan configuration should NOT be accessible by others.
(process:18222): WARNING **: 11:36:16.570: Permissions for /etc/netplan/01-network-manager-all.yaml are too open. Netplan configuration should NOT be accessible by others.
(process:18222): WARNING **: 11:36:16.671: Permissions for /etc/netplan/01-netcfg.yaml are too open. Netplan configuration should NOT be accessible by others.
(process:18222): WARNING **: 11:36:16.671: Permissions for /etc/netplan/01-network-manager-all.yaml are too open. Netplan configuration should NOT be accessible by others.
```

FIGURE 34 – `sudo netplan apply` — application de la configuration réseau

La configuration Netplan attribue l'adresse IP statique 172.17.8.10/24 à l'interface réseau du serveur, ainsi que la passerelle (172.17.8.1, SVI VLAN 40 du Catalyst). Cette adresse fixe est indispensable pour que le relais `ip helper-address` du Catalyst puisse toujours joindre le serveur, quel que soit l'état du réseau.

3.4.2 Configuration des pools DHCP

```
GNU nano 7.2 /etc/dhcp/dhcpd.conf
# Paramètres globaux
default-lease-time 86400;
max-lease-time 172800;
authoritative;

# VLAN 10 – Purchases (172.17.1.0/24)
subnet 172.17.1.0 netmask 255.255.255.0 {
    range 172.17.1.10 172.17.1.100;
    option routers 172.17.1.1;
    option domain-name-servers 172.17.8.10;
    option domain-name "almassar.local";
}

# VLAN 20 – Clients (172.17.4.0/24)
subnet 172.17.4.0 netmask 255.255.255.0 {
    range 172.17.4.10 172.17.4.100;
    option routers 172.17.4.1;
    option domain-name-servers 172.17.8.10;
    option domain-name "almassar.local";
}

# VLAN 30 – HR (172.17.6.0/24)
subnet 172.17.6.0 netmask 255.255.255.0 {
    range 172.17.6.10 172.17.6.100;
    option routers 172.17.6.1;
    option domain-name-servers 172.17.8.10;
    option domain-name "almassar.local";
}

# VLAN 40 – Services (déclaré mais pas de range – IP fixe uniquement)
subnet 172.17.8.0 netmask 255.255.255.0 {
    option routers 172.17.8.1;
    option domain-name-servers 172.17.8.10;
}
```

FIGURE 35 – Fichier `/etc/dhcp/dhcpd.conf` — pools pour chaque VLAN

```
$ sudo systemctl start isc-dhcp-server
sudo systemctl status isc-dhcp-server
● isc-dhcp-server.service - ISC DHCP IPv4 server
   Loaded: loaded (/usr/lib/systemd/system/isc-dhcp-server.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-04-13 18:02:41 +01; 20ms ago
     Docs: man:dhcpd(8)
    Main PID: 14444 (dhcpd)
      Tasks: 1 (limit: 18465)
    Memory: 4.6M (peak: 5.1M)
       CPU: 10ms
    CGroup: /system.slice/isc-dhcp-server.service
            └─14444 dhcpd -user dhcpd -group dhcpd -f -4 -pf /run/dhcp-server/dhcpd.pid -cf /etc/dhcp/dhcpd.conf enp108s0

Apr 13 18:02:41 dhcpd[14444]: All rights reserved.
Apr 13 18:02:41 dhcpd[14444]: For info, please visit https://www.isc.org/software/dhcp/
Apr 13 18:02:41 dhcpd[14444]: Config file: /etc/dhcp/dhcpd.conf
Apr 13 18:02:41 sh[14444]: Config file: /etc/dhcp/dhcpd.conf
Apr 13 18:02:41 sh[14444]: Database file: /var/lib/dhcp/dhcpd.leases
Apr 13 18:02:41 sh[14444]: PID file: /run/dhcp-server/dhcpd.pid
Apr 13 18:02:41 dhcpd[14444]: Database file: /var/lib/dhcp/dhcpd.leases
Apr 13 18:02:41 sh[14444]: Wrote 0 leases to leases file.
Apr 13 18:02:41 dhcpd[14444]: PID file: /run/dhcp-server/dhcpd.pid
Apr 13 18:02:41 dhcpd[14444]: Wrote 0 leases to leases file.
```

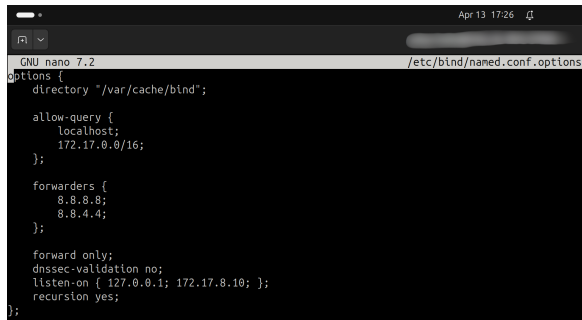
FIGURE 36 – État du service DHCP (`systemctl status isc-dhcp-server`)

Chaque pool attribue une plage d'adresses distincte par VLAN (172.17.1.10-100 pour VLAN 10, 172.17.4.10-100 pour VLAN 20, 172.17.6.10-100 pour VLAN 30), en indiquant à chaque client sa passerelle (`option routers`) et l'adresse du serveur DNS interne (`option domain-name-servers 172.17.8.10`).

3.5 Serveur Ubuntu — DNS (BIND9)

Le serveur DNS héberge la zone authoritative `almassar.local`, résolue vers l'IP du serveur (172.17.8.10).

3.5.1 Configuration de BIND9



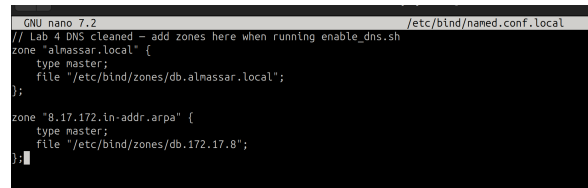
```
GNU nano 7.2 /etc/bind/named.conf.options
options {
    directory "/var/cache/bind";

    allow-query {
        localhost;
        172.17.0.0/16;
    };

    forwarders {
        8.8.8.8;
        8.8.4.4;
    };

    forward only;
    dnssec-validation no;
    listen-on { 127.0.0.1; 172.17.8.10; };
    recursion yes;
};
```

FIGURE 37 – named.conf.options — récursivité et écoute (any)



```
GNU nano 7.2 /etc/bind/named.conf.local
// Lab 4 DNS cleaned - add zones here when running enable_dns.sh
zone "almassar.local" {
    type master;
    file "/etc/bind/zones/db.almassar.local";
};

zone "8.17.172.in-addr.arpa" {
    type master;
    file "/etc/bind/zones/db.172.17.8";
};
```

FIGURE 38 – named.conf.local — déclaration de la zone almassar.local

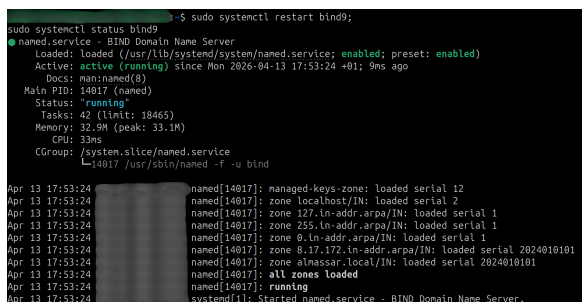


```
GNU nano 7.2 /etc/bind/zones/db.almassar.local *
$TTL 604800
@ IN SOA ns1.almassar.local. admin.almassar.local. (
    2024010101 ; Numéro de série
    604800 ; Refresh
    86400 ; Retry
    2419200 ; Expire
    604800 ) ; Negative Cache TTL

@ IN NS ns1.almassar.local.

ns1 IN A 172.17.8.10
server IN A 172.17.8.10
www IN A 172.17.8.10
ftp IN A 172.17.8.10
```

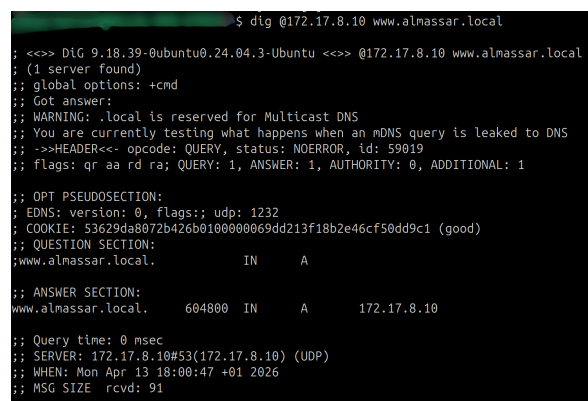
FIGURE 39 – Fichier de zone db.almassar.local — enregistrements A et NS



```
$ sudo systemctl restart bind9;
sudo systemctl status bind9
● named.service - BIND Domain Name Server
   Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-04-13 17:53:24 +01; 9ms ago
     Docs: man:named(8)
    Main PID: 14017 (named)
      Status: "running"
        Tasks: 42 (limit: 18465)
       Memory: 32.9M (peak: 33.1M)
          CPU: 33ms
    CGroup: /system.slice/named.service
            └─14017 /usr/sbin/named -f -u bind

Apr 13 17:53:24 named[14017]: managed-keys-zone: loaded serial 12
Apr 13 17:53:24 named[14017]: zone localhost/IN: loaded serial 2
Apr 13 17:53:24 named[14017]: zone 127.0.0.1/IN: loaded serial 1
Apr 13 17:53:24 named[14017]: zone 255.in-addr.arpa/IN: loaded serial 1
Apr 13 17:53:24 named[14017]: zone 0.in-addr.arpa/IN: loaded serial 1
Apr 13 17:53:24 named[14017]: zone 8.17.172.in-addr.arpa/IN: loaded serial 2024010101
Apr 13 17:53:24 named[14017]: zone almassar.local/IN: loaded serial 2024010101
Apr 13 17:53:24 named[14017]: all zones loaded
Apr 13 17:53:24 named[14017]: running
Apr 13 17:53:24 systemd[1]: Started named.service - BIND Domain Name Server.
```

FIGURE 40 – État du service BIND9 (systemctl status bind9)



```
$ dig @172.17.8.10 www.almassar.local

;<<> Dig 9.18.39-0ubuntu0.24.04.3-Ubuntu <<> @172.17.8.10 www.almassar.local
;; (1 server found)
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
;; -->HEADER<<- opcode: QUERY, status: NOERROR, id: 59019
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 53629da8072b426b0100000069dd213f18b2e46cf50dd9c1 (good)
;; QUESTION SECTION:
;www.almassar.local.                IN      A

;; ANSWER SECTION:
www.almassar.local. 604800 IN      A      172.17.8.10

;; Query time: 0 msec
;; SERVER: 172.17.8.10#53(172.17.8.10) (UDP)
;; WHEN: Mon Apr 13 18:00:47 +01 2026
;; MSG SIZE rcvd: 91
```

FIGURE 41 – dig @172.17.8.10 www.almassar.local — résolution correcte


```
# Bloc 1 – Intercepte HTTP (port 80) et redirige vers HTTPS
server {
    listen 80;
    server_name www.almassar.local almassar.local 172.17.8.10;
    return 301 https://$host$request_uri;
}

# Bloc 2 – Sert les pages en HTTPS (port 443)
server {
    listen 443 ssl;
    server_name www.almassar.local almassar.local 172.17.8.10;

    ssl_certificate /etc/nginx/ssl/almassar.crt;
    ssl_certificate_key /etc/nginx/ssl/almassar.key;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    root /var/www/almassar;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

FIGURE 47 – Redirection HTTP → HTTPS dans la config Nginx

Toute requête HTTP (port 80) est redirigée en 301 Moved Permanently vers HTTPS (port 443), garantissant que la communication est toujours chiffrée. Le test avec curl sur le port 80 confirme la redirection, et l'accès direct au port 443 retourne bien la page Al Massar avec le certificat TLS :

```
$ curl -k https://127.0.0.1
curl -k https://www.almassar.local
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>ALMASSAR - Portail Interne</title>
</head>
<body>
  <h1>ALMASSAR Company</h1>
  <p>Portail web interne – Connexion HTTPS active</p>
  <p>Serveur : 172.17.8.10 | VLAN 40 - Services</p>
</body>
</html>
```

FIGURE 49 – Test HTTP avant redirection (accès port 80)

```
$ sudo systemctl status nginx
nginx.service - A high performance web server and a reverse proxy server
Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
Active: active (running) since Tue 2026-04-14 15:48:28 +01; 46s ago
Docs: man:nginx(8)
Main PID: 5982 (nginx)
Tasks: 21 (limit: 18442)
Memory: 15.7M (peak: 17.3M)
CPU: 52ms
CGroup: /systemd/system/nginx.service
├─5982 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
├─5983 "nginx: worker process"
├─5984 "nginx: worker process"
├─5985 "nginx: worker process"
├─5986 "nginx: worker process"
├─5987 "nginx: worker process"
├─5988 "nginx: worker process"
├─5989 "nginx: worker process"
├─5990 "nginx: worker process"
├─5991 "nginx: worker process"
├─5992 "nginx: worker process"
├─5993 "nginx: worker process"
├─5994 "nginx: worker process"
├─5995 "nginx: worker process"
├─5996 "nginx: worker process"
├─5997 "nginx: worker process"
├─5998 "nginx: worker process"
├─5999 "nginx: worker process"
├─6000 "nginx: worker process"
├─6001 "nginx: worker process"
├─6002 "nginx: worker process"
└─6003 "nginx: worker process"
```

FIGURE 48 – Test du statut après configuration HTTPS

```
$ curl -v http://172.17.8.10 2>&1 | grep "< HTTP\Location"
* HTTP/1.1 301 Moved Permanently
* Location: https://172.17.8.10/
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL
* subject:
* issuer:
* TLSv1.3 (IN), TLS handshake, NewSession Ticket (4):
* TLSv1.3 (IN), TLS handshake, NewSession Ticket (4):
```

FIGURE 50 – Justification de la sécurisation TLS (certificat affiché)

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>ALMASSAR - Portail Interne</title>
</head>
<body>
  <h1>ALMASSAR Company</h1>
  <p>Portail web interne – Connexion HTTPS active</p>
  <p>Serveur : 172.17.8.10 | VLAN 40 - Services</p>
</body>
</html>
```

FIGURE 51 – Page web Al Massar servie en HTTPS depuis un poste client

3.8 Tests de connectivité physique

3.8.1 Vérification inter-VLAN

```
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet 2:

    Connection-specific DNS Suffix . : almassar.local
    IPv4 Address. . . . . : 172.17.1.12
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 172.17.1.1
```

FIGURE 57 – Ping inter-VLAN depuis un hôte VLAN 10 (HP#1)

```
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet 2:

    Connection-specific DNS Suffix . : almassar.local
    IPv4 Address. . . . . : 172.17.4.10
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 172.17.4.1
```

FIGURE 58 – Ping inter-VLAN depuis un hôte VLAN 20 (HP#1)

```
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet 2:

    Connection-specific DNS Suffix . : almassar.local
    IPv4 Address. . . . . : 172.17.6.10
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 172.17.6.1
```

FIGURE 59 – Ping inter-VLAN depuis un hôte VLAN 30 (HP#2)

Les pings inter-VLAN valident le routage L3 du Catalyst 3650 : un paquet issu du VLAN 10 (172.17.1.x) atteint un hôte en VLAN 30 (172.17.6.x) en traversant les SVIs du Catalyst. Le délai de réponse (inférieur à 2 ms en LAN) confirme que le routage est effectué localement par le Catalyst, sans passer par le routeur. La figure suivante illustre cette communication bidirectionnelle :

```
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\>ping 172.17.1.1
PING 172.17.1.1 (172.17.1.1) 56(84) bytes of data:
64 bytes from 172.17.1.1: icmp_seq=1 ttl=255 time=2.40 ms
64 bytes from 172.17.1.1: icmp_seq=2 ttl=255 time=2.63 ms
64 bytes from 172.17.1.1: icmp_seq=3 ttl=255 time=2.71 ms
64 bytes from 172.17.1.1: icmp_seq=4 ttl=255 time=4.09 ms

--- 172.17.1.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 2.401/2.957/4.091/0.664 ms

C:\>ping 172.17.4.1
PING 172.17.4.1 (172.17.4.1) 56(84) bytes of data:
64 bytes from 172.17.4.1: icmp_seq=1 ttl=255 time=2.51 ms
64 bytes from 172.17.4.1: icmp_seq=2 ttl=255 time=2.46 ms
64 bytes from 172.17.4.1: icmp_seq=3 ttl=255 time=2.46 ms
64 bytes from 172.17.4.1: icmp_seq=4 ttl=255 time=2.67 ms

--- 172.17.4.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 2.460/2.525/2.668/0.084 ms

C:\>ping 172.17.6.1
PING 172.17.6.1 (172.17.6.1) 56(84) bytes of data:
64 bytes from 172.17.6.1: icmp_seq=1 ttl=255 time=2.41 ms
64 bytes from 172.17.6.1: icmp_seq=2 ttl=255 time=2.44 ms
64 bytes from 172.17.6.1: icmp_seq=3 ttl=255 time=2.48 ms

--- 172.17.6.1 ping statistics ---
4 packets transmitted, 3 received, 25% packet loss, time 3004ms
rtt min/avg/max/mdev = 2.412/2.443/2.478/0.027 ms

C:\>ping 172.17.8.1
PING 172.17.8.1 (172.17.8.1) 56(84) bytes of data:
64 bytes from 172.17.8.1: icmp_seq=1 ttl=255 time=2.41 ms
64 bytes from 172.17.8.1: icmp_seq=2 ttl=255 time=2.30 ms
64 bytes from 172.17.8.1: icmp_seq=3 ttl=255 time=2.59 ms
64 bytes from 172.17.8.1: icmp_seq=4 ttl=255 time=2.63 ms

--- 172.17.8.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3002ms
rtt min/avg/max/mdev = 2.296/2.478/2.626/0.133 ms
```

FIGURE 60 – Preuve de communication inter-VLAN (VLAN 10 ↔ VLAN 30)

3.8.2 Vérification DNS

```
C:\Users\shop>nslookup www.almassar.local
Server: UnKnown
Address: 172.17.8.10

Name:    www.almassar.local
Address: 172.17.8.10
```

FIGURE 61 – Résolution DNS depuis un client (nslookup almassar.local)

```
Wireless LAN adapter Wi-Fi 2:
Media State . . . . . : Media diso
Connection-specific DNS Suffix . . : ulr.ac.ma
Ethernet adapter Connexion réseau Bluetooth:
Media State . . . . . : Media diso
Connection-specific DNS Suffix . . :
Ethernet adapter Ethernet 2:
Media State . . . . . : Media diso
Connection-specific DNS Suffix . . :
C:\Windows\system32\cmd.exe
Windows IP Configuration

@nslookup www.almassar.local
Server: UnKnown
Address: 172.17.8.10
Name: www.almassar.local
Address: 172.17.8.10
Unknown adapter x-ovpn-tap:
Media State . . . . . : Media diso
Connection-specific DNS Suffix . . :
IPV4 Address. . . . . : 172.17.4.18
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . : 172.17.4.1
```

FIGURE 62 – Vérification avancée (dig almassar.local)

```
C:\Users\shop>nslookup 172.17.8.10
Server: ns1.almassar.local
Address: 172.17.8.10

Name:    www.almassar.local
Address: 172.17.8.10
```

FIGURE 63 – Résolution DNS inverse (PTR)

L'ensemble de la couche physique est donc pleinement opérationnel : routage inter-VLAN via le Catalyst 3650, accès Internet via NAT sur le routeur Cisco 2900, et quatre services applicatifs (DHCP, DNS, HTTPS, FTPS) fournis par un unique serveur Ubuntu. La partie suivante présente la réplique de cette même infrastructure en environnement 100% logiciel, sans aucun matériel dédié.

4 Partie II — Déploiement Virtuel (Linux Network Namespaces)

4.1 Concept et Architecture

La seconde approche reproduit **entièrement la même infrastructure logique** sans aucun matériel ni machine virtuelle lourde. Elle exploite les primitives Linux suivantes :

Primitive Linux	Équivalent réseau physique
<code>ip netns</code> (Network Namespace)	Machine / équipement réseau isolé
<code>ip link add ... type bridge</code>	Switch L2 (VLAN-aware)
<code>ip link add ... type veth peer</code>	Câble réseau (virtuel)
<code>bridge vlan add (802.1Q)</code>	Port trunk / access VLAN
<code>iptables MASQUERADE</code>	NAT overload (PAT)
<code>ip netns exec ubuntu ...</code>	Console serveur Ubuntu

4.1.1 Namespaces créés

Listing 3 – CREATE_NAMESPACES.sh — création des namespaces réseau

```
1 for ns in router sw-core ubuntu pc-vlan10 pc-vlan30; do
2     ip netns add $ns
3 done
4 for ns in router sw-core ubuntu pc-vlan10 pc-vlan30; do
5     ip netns exec $ns ip link set lo up
6 done
```

Namespace	Rôle	Adresse(s)
router	Routeur bordure / passerelle WAN	10.0.0.1/24, 192.168.99.2/30
sw-core	Switch L3 / routage inter-VLAN	10.0.0.2/24, br-sw1, br-sw2
ubuntu	Serveur DHCP/DNS/HTTPS/FTPS	172.17.4.10/24
pc-vlan10	Poste client VLAN 10	172.17.1.10/24 (statique)
pc-vlan30	Poste client VLAN 30	172.17.3.10/24 (statique)

4.1.2 Plan d’adressage virtuel

VLAN	Département	Réseau	Passerelle (SVI bridge)
10	Purchases	172.17.1.0/24	172.17.1.1 (br-sw1)
30	HR	172.17.3.0/24	172.17.3.1 (br-sw2)
40	Services	172.17.4.0/24	172.17.4.1 (br-sw2)

4.2 Interface TUI — CONF_WINDOW.sh

Avant le déploiement, un écran d’accueil interactif (**whiptail**) guide l’opérateur à travers trois étapes : confirmation du déploiement, identification de l’ingénieur, activation du bouclier de sécurité (ACL).

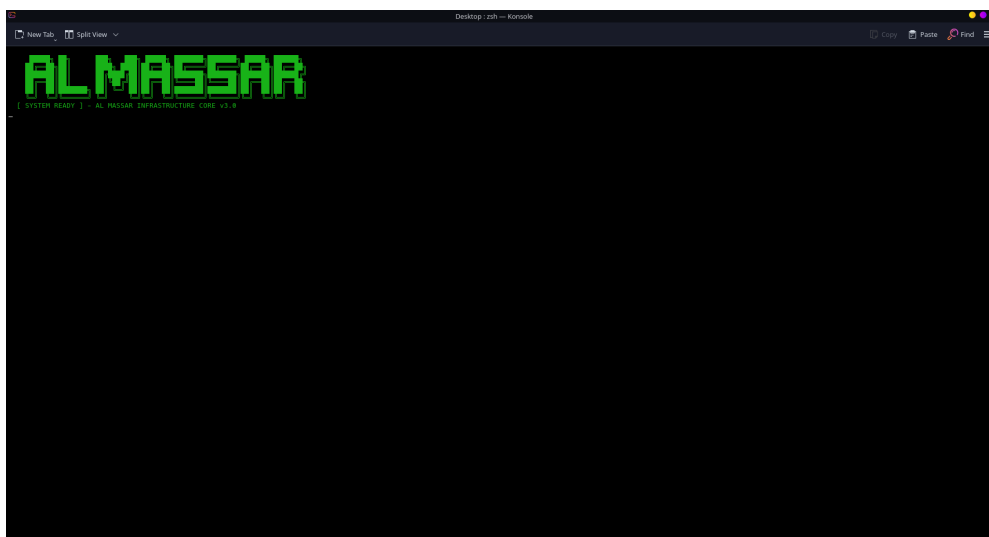


FIGURE 64 – Logo ASCII Al Massar affiché dans le terminal au lancement

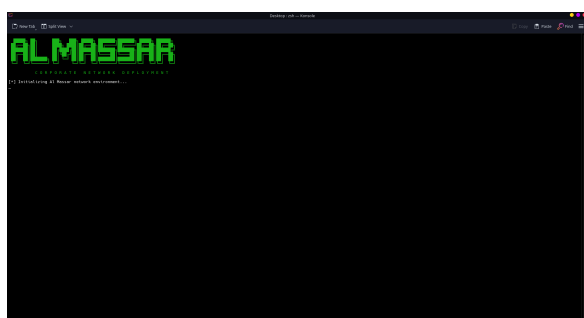


FIGURE 65 – Fenêtre d'état système (TUI whiptail)

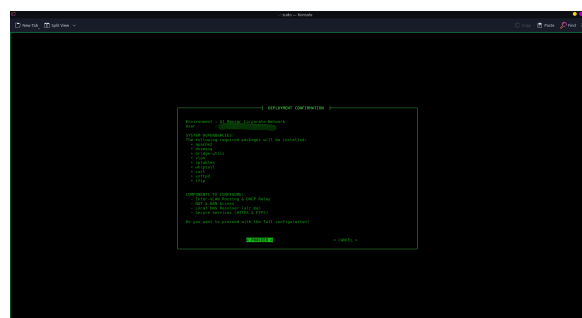


FIGURE 66 – Confirmation du déploiement (PROCEED / CANCEL)

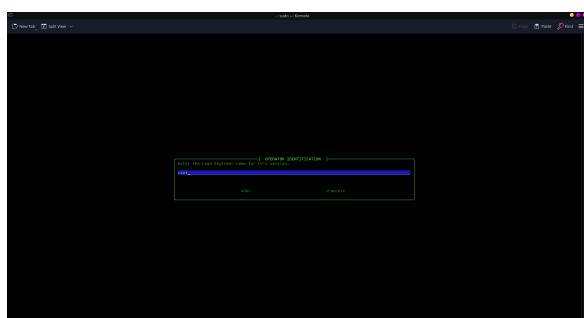


FIGURE 67 – Saisie du nom de l'ingénieur responsable

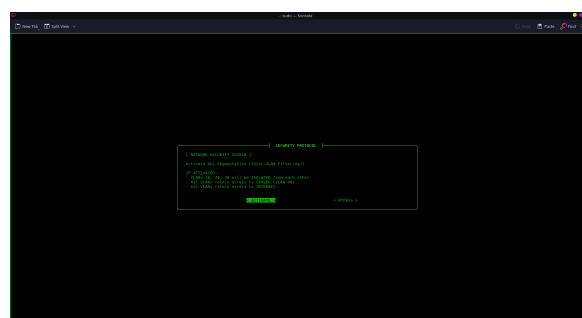


FIGURE 68 – Activation optionnelle du bouclier ACL (segmentation inter-VLAN)

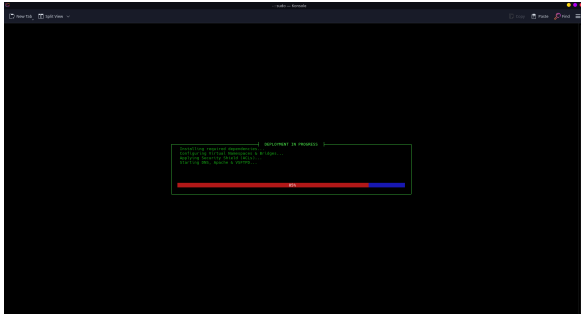


FIGURE 69 – Barre de progression pendant l’installation des dépendances

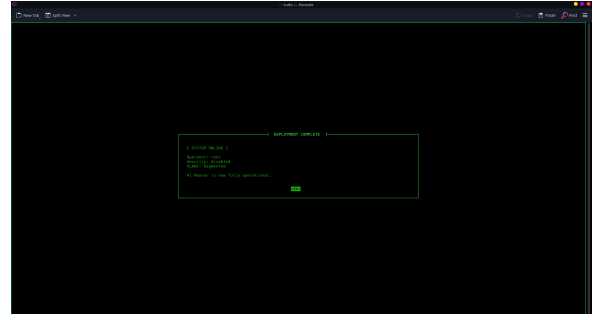


FIGURE 70 – Fenêtre de fin de déploiement (succès + fichier log)

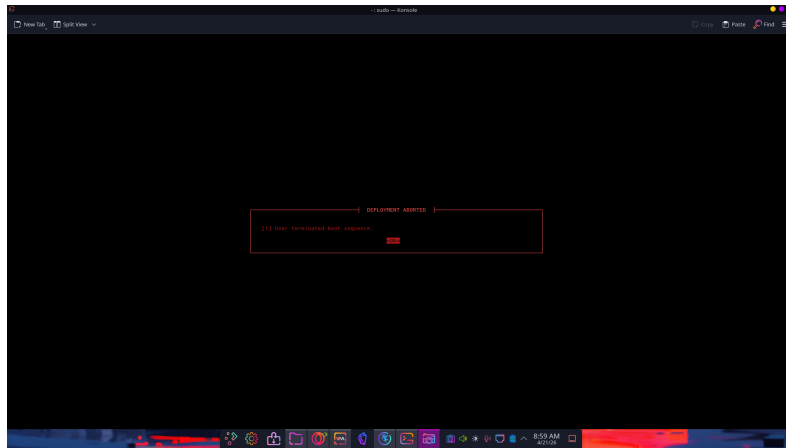


FIGURE 71 – Annulation du déploiement (bouton CANCEL)

Un fichier de log est généré dans `LOG_AL_MASSAR/deployment.log` avec l’horodatage, le nom de l’opérateur et le mode de sécurité choisi :

Listing 4 – Extrait de `LOG_AL_MASSAR/deployment.log`

```
[2026-04-13 15:33:21] - Operator: hathouti | Security: ENABLED |
Status: SUCCESS
```

4.3 Orchestrateur Principal — BOOT.sh

`BOOT.sh` est le point d’entrée unique. Il orchestre l’ensemble des scripts dans un ordre précis : fondations L1/L2, câblage, logique VLAN, transit & NAT, puis services.

Listing 5 – `BOOT.sh` — séquence de déploiement complète

```
1 #!/usr/bin/env bash
2 set -euo pipefail
3
4 # --- CLEANUP, DEPENDENCIES CHECK & TUI ---
5 sudo ./CONF_WINDOW.sh
6 sudo ./CLEANUP.sh
7 sudo ./DEPENDENCIES.sh
8
```

```

9 # 1. FOUNDATIONS (L1/L2)
10 sudo ./CREATE_NAMESPACES.sh
11 sudo ./CREATE_L2_SWITCHES.sh
12
13 # 2. CABLING
14 sudo ./PCs.sh
15 sudo ./UBUNTU_SERVER.sh
16
17 # 3. VLAN LOGIC (Configure BEFORE services)
18 sudo ./CREATE_VLANS.sh
19
20 # 4. TRANSIT & NAT
21 sudo ./CONNECT_CORE_TO_ROUTER.sh
22 sudo ./SETUP_NAT.sh
23
24 # 5. SERVICE CONFIGURATION AND STARTUP
25 sudo ./UBUNTU_SERV_SCRIPTS/SETUP_HTTPS.sh
26 sudo ./UBUNTU_SERV_SCRIPTS/SETUP_DNS.sh
27 sudo ./UBUNTU_SERV_SCRIPTS/SETUP_FTPS.sh
28 sudo ./UBUNTU_SERV_SCRIPTS/SETUP_DHCP.sh

```

```

testuser@testuser-lubuntu-vm: ~/Desktop/AL_MASSAR
testuser@testuser-lubuntu-vm: ~/Desktop/AL_MASSAR$ ls
BOOT.sh          CREATE_L2_SWITCHES.sh  deployment.log      SETUP_NAT.sh
CLEANUP.sh        CREATE_NAMESPACES.sh  LAUNCH_SERVICES.sh TESTS
CONF_WINDOW.sh    CREATE_VLANS.sh        PCs.sh              UBUNTU_SERVER.sh
CONNECT_CORE_TO_ROUTER.sh  DEPENDENCIES.sh      README.md           UBUNTU_SERV_SCRIPTS
testuser@testuser-lubuntu-vm: ~/Desktop/AL_MASSAR$ sudo ./LAUNCH_SERVICES.sh
[sudo] password for testuser:
=====
[ ] LAUNCHING AL MASSAR NETWORK SERVICES
=====
[1/4] Configuring Return Routes for Ubuntu Server...
[2/4] Enforcing DNS Resolver for pc-vlan10...
[3/4] Starting Apache Web Server (SSL Enabled)...
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Module socache_shmcb already enabled
Module ssl already enabled
Site default-ssl already enabled
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the
'ServerName' directive globally to suppress this message
[4/4] Starting VSFTPD Service (Explicit TLS)...
=====
✓ ALL SERVICES SUCCESSFULLY LAUNCHED AND SECURED
=====
testuser@testuser-lubuntu-vm: ~/Desktop/AL_MASSAR$

```

FIGURE 72 – Lancement de BOOT.sh — démarrage des services dans la console

L'ordre des scripts est rigoureux et non interchangeable : les bridges et les namespaces doivent exister avant le câblage, le câblage avant la logique VLAN, et la logique VLAN avant les services. Toute inversion produit des erreurs silencieuses (interfaces inexistantes, adresses non routables). La figure suivante montre l'état de l'infrastructure réseau juste après déploiement, avant le démarrage des services applicatifs :

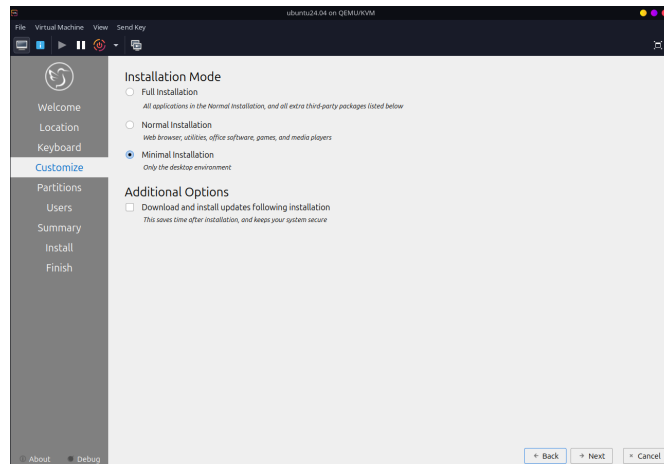


FIGURE 73 – Résultat minimal après déploiement (namespaces et bridges actifs)

On observe les cinq namespaces créés et actifs, ainsi que les bridges Linux **br-sw1** et **br-sw2** dans le namespace **sw-core**. La couche réseau est prête : les sections suivantes décrivent chaque script de la séquence en détail.

4.4 Infrastructure L2 — Bridges et Câblage

4.4.1 CREATE_L2_SWITCHES.sh — Création des bridges

Deux bridges Linux (**br-sw1** et **br-sw2**) émulent les switches d'accès. Le STP est désactivé pour éviter le délai de 30 secondes en lab.

Listing 6 – CREATE_L2_SWITCHES.sh — bridges VLAN-aware

```

1 ip netns exec sw-core ip link add br-sw1 type bridge
2 ip netns exec sw-core ip link add br-sw2 type bridge
3
4 # Disable STP (Avoids the 30s wait)
5 ip netns exec sw-core brctl stp br-sw1 off 2>/dev/null || true
6 ip netns exec sw-core brctl stp br-sw2 off 2>/dev/null || true
7
8 ip netns exec sw-core ip link set br-sw1 up
9 ip netns exec sw-core ip link set br-sw2 up

```

4.4.2 PCs.sh — Câblage des postes clients

Chaque PC est connecté via une paire *veth* au bridge correspondant. Le port switch est immédiatement taggé en mode *access* (untagged) pour le VLAN approprié.

Listing 7 – PCs.sh — Connexion de pc-vlan10 et pc-vlan30

```

1 # PC 1 : VLAN 10 (Purchasing)
2 ip link add veth-pc10 type veth peer name veth-sw10
3 ip link set veth-sw10 netns sw-core
4 ip netns exec sw-core ip link set veth-sw10 master br-sw1
5 ip netns exec sw-core ip link set veth-sw10 up

```

```

6 ip netns exec sw-core bridge vlan add dev veth-sw10 vid 10 pvid
  untagged
7
8 ip link set veth-pc10 netns pc-vlan10
9 ip netns exec pc-vlan10 ip addr add 172.17.1.10/24 dev veth-pc10
10 ip netns exec pc-vlan10 ip link set veth-pc10 up
11 ip netns exec pc-vlan10 ip route add default via 172.17.1.1
12
13 # PC 2 : VLAN 30 (HR)
14 ip link add veth-pc30 type veth peer name veth-sw30
15 # ... (idem avec vid 30 et 172.17.3.10/24)

```

4.4.3 UBUNTU_SERVER.sh — Câblage du serveur Ubuntu

Listing 8 – UBUNTU_SERVER.sh — connexion du serveur au VLAN 40

```

1 ip link add veth-ubuntu type veth peer name veth-sw40
2 ip link set veth-sw40 netns sw-core
3 ip netns exec sw-core ip link set veth-sw40 master br-sw2
4 ip netns exec sw-core ip link set veth-sw40 up
5 ip netns exec sw-core bridge vlan add dev veth-sw40 vid 40 pvid
  untagged
6
7 ip link set veth-ubuntu netns ubuntu
8 ip netns exec ubuntu ip addr add 172.17.4.10/24 dev veth-ubuntu
9 ip netns exec ubuntu ip link set veth-ubuntu up
10 ip netns exec ubuntu ip route add default via 172.17.4.1

```

4.5 Logique VLAN — CREATE_VLANS.sh

Le VLAN filtering est activé sur les bridges, puis des adresses SVI sont assignées directement sur les bridges (équivalent des interfaces VLAN du Catalyst). Le flag **self untagged** sur le port CPU est essentiel : sans lui, les trames VLAN arrivent taguées à la pile IP du noyau, qui les ignore silencieusement (*ARP failed*).

Listing 9 – CREATE_VLANS.sh — filtering, SVIs et ip_forward

```

1 # Enable VLAN filtering
2 ip netns exec sw-core ip link set dev br-sw1 type bridge
  vlan_filtering 1
3 ip netns exec sw-core ip link set dev br-sw2 type bridge
  vlan_filtering 1
4
5 # FIX: 'self_untagged' required on the bridge CPU port
6 ip netns exec sw-core bridge vlan add dev br-sw1 vid 10 self
  untagged
7 ip netns exec sw-core bridge vlan add dev br-sw1 vid 20 self
  untagged
8 ip netns exec sw-core bridge vlan add dev br-sw2 vid 30 self
  untagged

```

```

9 ip netns exec sw-core bridge vlan add dev br-sw2 vid 40 self
   untagged
10
11 # SVI gateways
12 ip netns exec sw-core ip addr add 172.17.1.1/24 dev br-sw1
13 ip netns exec sw-core ip addr add 172.17.3.1/24 dev br-sw2
14 ip netns exec sw-core ip addr add 172.17.4.1/24 dev br-sw2
15
16 ip netns exec sw-core sysctl -w net.ipv4.ip_forward=1

```

Subtilité critique — self untagged sur le port CPU

Sans l'option `self untagged` sur le bridge lui-même, les trames VLAN remontent taguées vers la pile IP du noyau Linux. La pile IP ne sait pas les traiter et les jette silencieusement : les ARP ne reçoivent aucune réponse, et tous les pings retournent *Destination Host Unreachable*. Ce bug subtil a nécessité une analyse approfondie pour être identifié.

Une fois le VLAN filtering activé et les SVIs configurées, le namespace `sw-core` se comporte exactement comme le Catalyst 3650 en production : il achemine les paquets entre les VLANs grâce à `ip_forward=1` et aux routes locales sur les bridges.

4.6 Transit L3 et NAT — CONNECT_CORE_TO_ROUTER.sh & SETUP_NAT.sh

4.6.1 Lien transit sw-core ↔ router

Listing 10 – CONNECT_CORE_TO_ROUTER.sh — lien point-à-point L3

```

1 # Transit veth pair
2 ip link add veth-sw2r type veth peer name veth-r2sw
3
4 # Switch side (10.0.0.2)
5 ip link set veth-sw2r netns sw-core
6 ip netns exec sw-core ip addr add 10.0.0.2/24 dev veth-sw2r
7
8 # Router side (10.0.0.1)
9 ip link set veth-r2sw netns router
10 ip netns exec router ip addr add 10.0.0.1/24 dev veth-r2sw
11
12 # Return route (router -> internal VLANs via sw-core)
13 ip netns exec router ip route replace 172.17.0.0/16 via 10.0.0.2

```

4.6.2 NAT et accès Internet

Listing 11 – SETUP_NAT.sh — MASQUERADE via l'interface WAN de l'hôte

```

1 WAN_IF=$(ip route | grep default | awk '{print $5}' | head -n1)
2
3 # Host <-> router transit link

```



```

4 ip link add veth-host-wan type veth peer name veth-router-wan
5 ip link set veth-router-wan netns router
6
7 ip addr replace 192.168.99.1/30 dev veth-host-wan
8 ip netns exec router ip addr replace 192.168.99.2/30 dev veth-
  router-wan
9
10 # Default route of the router namespace
11 ip netns exec router ip route replace default via 192.168.99.1
12
13 # MASQUERADE on the host WAN interface
14 iptables -t nat -A POSTROUTING -o "$WAN_IF" -j MASQUERADE
15
16 # FIX: INSERT at position 1 (libvirt/KVM rules block at end of
  FORWARD)
17 iptables -I FORWARD 1 -i veth-host-wan -o "$WAN_IF" -j ACCEPT
18 iptables -I FORWARD 1 -i "$WAN_IF" -o veth-host-wan \
19   -m state --state RELATED,ESTABLISHED -j ACCEPT

```

Architecture NAT virtuelle

Le namespace **router** n'a aucun accès direct à Internet. On crée un lien dédié **host** ↔ **router** (192.168.99.0/30). Le MASQUERADE est appliqué sur le côté **hôte** (seul connecté au réseau réel). Le trafic suit donc la chaîne :
 pc-vlan10 → sw-core → router → host → WAN (Internet)

Les règles iptables sont insérées en **position 1** de la chaîne FORWARD (-I FORWARD 1) plutôt qu'en fin de chaîne (-A) : dans un environnement libvirt/KVM, des règles bloquantes sont automatiquement ajoutées en fin de chaîne par libvirtd, et un simple -A les laisserait prendre effet avant les règles permissives. L'insertion en tête garantit que le trafic légitime est traité en priorité.

4.7 Services du Serveur Ubuntu Virtuel

4.7.1 DHCP — SETUP_DHCP.sh

Listing 12 – SETUP_DHCP.sh — configuration des pools DHCP multi-VLAN

```

1 cat <<EOF | ip netns exec ubuntu tee /etc/dhcp/dhcpd.conf
2 default -lease-time 600;
3 max-lease-time 7200;
4 authoritative;
5
6 # VLAN 10 (Purchasing)
7 subnet 172.17.1.0 netmask 255.255.255.0 {
8   range 172.17.1.50 172.17.1.100;
9   option routers 172.17.1.1;
10  option domain-name-servers 172.17.4.10;
11 }
12
13 # VLAN 30 (HR)

```

```

14 subnet 172.17.3.0 netmask 255.255.255.0 {
15     range 172.17.3.50 172.17.3.100;
16     option routers 172.17.3.1;
17     option domain-name-servers 172.17.4.10;
18 }
19 EOF
20 ip netns exec ubuntu service isc-dhcp-server restart

```

Le relais DHCP est assuré par `dhcrelay` dans le namespace `sw-core`, écoutant sur les bridges SVI et relayant vers le serveur Ubuntu (172.17.4.10).

Listing 13 – DHCP_RELAY_AGENT.sh — relais DHCP inter-VLAN

```

1 ip netns exec sw-core /usr/sbin/dhcrelay \
2     -i br-sw1 \
3     -i br-sw2 \
4     172.17.4.10 &

```

Le `&` final est intentionnel : `dhcrelay` est un processus bloquant et doit tourner en arrière-plan pour ne pas interrompre la séquence `BOOT.sh`. Il écoute sur les deux bridges SVI et redirige chaque requête DHCP broadcast en unicast vers le serveur Ubuntu. Le test du cycle DORA (Discover → Offer → Request → Acknowledge) confirme son bon fonctionnement :

```

testuser@testuser-lubuntu-vm: ~/Desktop/AL_MASSAR$ sudo ip netns exec pc-vlan10 dhclient -v veth-pc10
Internet Systems Consortium DHCP Client 4.4.3-P1
Copyright 2004-2022 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/

Listening on LPF/veth-pc10/ae:2c:fd:79:9d:74
Sending on   LPF/veth-pc10/ae:2c:fd:79:9d:74
Sending on   Socket/fallback
xid: warning: no netdev with useable HWADDR found for seed's uniqueness enforcement
xid: rand init seed (0x69982617) built using gethostid
DHCPDISCOVER on veth-pc10 to 255.255.255.255 port 67 interval 3 (xid=0xfe16a75f)
DHCPOFFER of 172.17.1.50 from 172.17.1.1
DHCPREQUEST for 172.17.1.50 on veth-pc10 to 255.255.255.255 port 67 (xid=0x5fa716fe)
DHCPACK of 172.17.1.50 from 172.17.1.1 (xid=0xfe16a75f)
Setting LLNMR support level "yes" for "5", but the global support level is "no".
Failed to set LLNMR configuration: Link 5 not known
Failed to set MulticastDNS configuration: Link 5 not known
Failed to set domain configuration: Link 5 not known
Failed to set DNS configuration: Link 5 not known
bound to 172.17.1.50 -- renewal in 240 seconds.
testuser@testuser-lubuntu-vm: ~/Desktop/AL_MASSAR$

```

FIGURE 74 – Test DHCP virtuel — cycle DORA complet (DISCOVER → OFFER → REQUEST → ACK)

4.7.2 DNS — SETUP_DNS.sh

Le serveur DNS BIND9 héberge la zone `uir.ma` (contrairement à la version physique qui utilisait `almassar.local`).

Listing 14 – SETUP_DNS.sh — zone uir.ma et enregistrements A

```

1 # Zone declaration
2 cat <<'EOF' | ip netns exec ubuntu tee /etc/bind/named.conf.local
3 zone "uir.ma" { type master; file "/etc/bind/db.uir.ma"; };
4 EOF
5
6 # Zone file
7 cat <<'EOF' | ip netns exec ubuntu tee /etc/bind/db.uir.ma
8 $TTL 604800
9 @ IN SOA srv.uir.ma. admin.uir.ma. ( 2 604800 86400 2419200 604800
10 )
11 @ IN NS srv.uir.ma.
12 @ IN A 172.17.4.10
13 srv IN A 172.17.4.10
14 www IN A 172.17.4.10
15 EOF
16
17 # Force IPv4 (namespace constraint)
18 sudo ip netns exec ubuntu sh -c 'echo "OPTIONS=\"-u_bind-4\">
19 etc/default/bind9'
20 ip netns exec ubuntu service bind9 restart

```

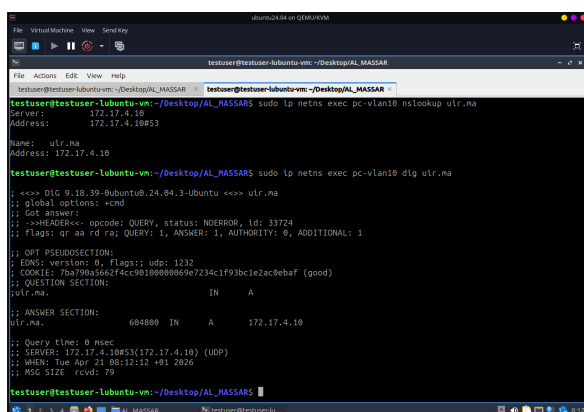


FIGURE 75 – nslookup uir.ma — résolution DNS virtuelle

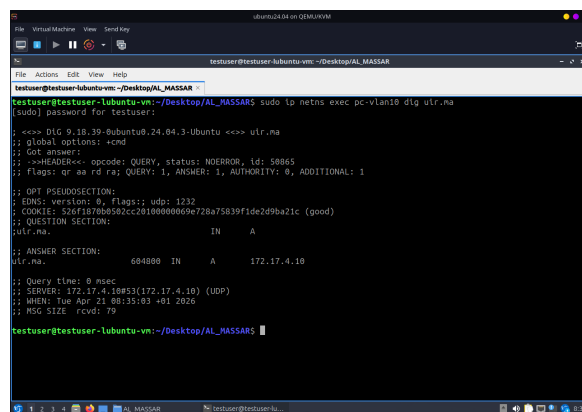


FIGURE 76 – dig uir.ma — inspection des enregistrements DNS

La commande **dig** confirme la présence des enregistrements A (**uir.ma** → **172.17.4.10**), de l'enregistrement NS (**srv.uir.ma**) et des paramètres SOA. La résolution est instantanée (Query time : 0 ms), preuve que BIND9 répond depuis son adresse dans le namespace **ubuntu**. L'option **-4** dans **/etc/default/bind9** est nécessaire pour éviter les erreurs d'écoute IPv6 dans un namespace sans IPv6.

4.7.3 HTTPS — SETUP_HTTPS.sh

En version virtuelle, **Apache2** est utilisé à la place de **Nginx**. Un certificat auto-signé est généré via **OpenSSL**, et la même page HTML *Al Massar* est déployée.

Listing 15 – SETUP_HTTPS.sh — Apache2 avec SSL auto-signé

```

1 ip netns exec ubuntu a2enmod ssl
2 ip netns exec ubuntu a2ensite default-ssl
3
4 # Certificat auto-signé (réutilisé aussi par vsftpd)
5 ip netns exec ubuntu openssl req -x509 -nodes -days 365 -newkey rsa
:2048 \
6     -keyout /etc/ssl/private/apache-selfsigned.key \
7     -out /etc/ssl/certs/apache-selfsigned.crt \
8     -subj "/C=MA/ST=Rabat/L=Rabat/O=UIR/OU=Engineering/CN=srv.uir.
ma"
9
10 ip netns exec ubuntu sed -i \
11     's|ssl-cert-snakeoil.pem|apache-selfsigned.crt|g' \
12     /etc/apache2/sites-available/default-ssl.conf
13
14 ip netns exec ubuntu service apache2 restart

```

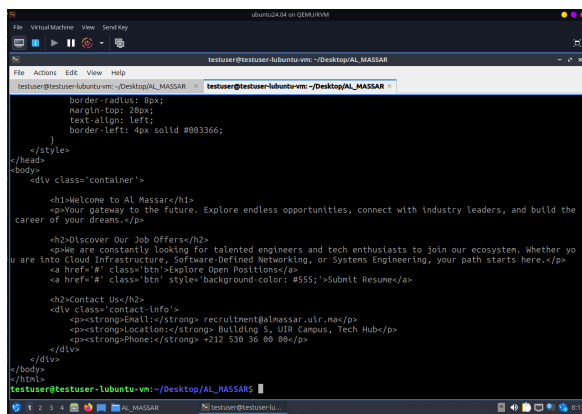


FIGURE 77 – `curl -k https://uir.ma` — réponse HTML du serveur

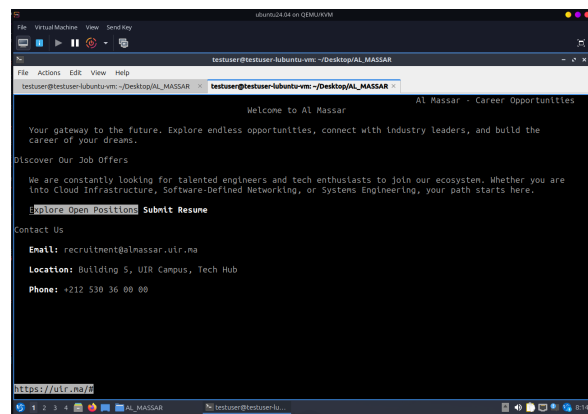


FIGURE 78 – Navigateur TUI `links` — rendu visuel de la page Al Massar

Le certificat auto-signé est accepté par `curl -k` et par le navigateur `links`. La chaîne complète est validée : DNS résout `uir.ma`, Apache répond en TLS sur le port 443, et la page HTML Al Massar s'affiche correctement. Le même certificat sera réutilisé par `vsftpd` pour chiffrer le canal FTP.

4.7.4 FTPS — `SETUP_FTPS.sh`

`vsftpd` est configuré en mode TLS explicite (**FTPS**), en mode passif avec une plage de ports dédiée (40000-40100) pour faciliter la traversée des bridges.

Listing 16 – `SETUP_FTPS.sh` — configuration `vsftpd` (extrait)

```

1 # User creation
2 ip netns exec ubuntu useradd -m -s /bin/bash testuser
3 ip netns exec ubuntu sh -c 'echo "testuser:123"|chpasswd'
4
5 # vsftpd.conf (extrait)
6 # ssl_enable=YES
7 # force_local_data_ssl=YES

```

```

8 # force_local_logins_ssl=YES
9 # rsa_cert_file=/etc/ssl/certs/apache-selfsigned.crt
10 # pasv_enable=YES
11 # pasv_min_port=40000 / pasv_max_port=40100
12 # pasv_address=172.17.4.10

```

Le mode passif (`pasv_enable=YES`) est indispensable en environnement virtualisé : en mode actif, le serveur tenterait d'initier une connexion retour vers le client, ce qui échoue systématiquement derrière un bridge ou un NAT. En mode passif, c'est toujours le client qui initie les deux connexions (contrôle sur port 21, données sur un port dans la plage 40000–40100), ce qui traverse naturellement les règles iptables :

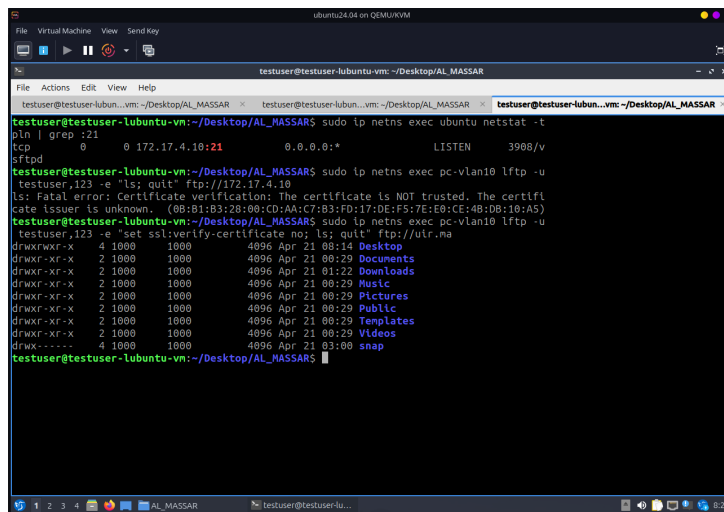


FIGURE 79 – Connexion FTPS depuis pc-vlan10 (lftp avec TLS forcé)

4.8 ACL et Segmentation — ACL.sh

Le bouclier ACL (optionnel, activé via la TUI) isole les VLANs clients entre eux tout en autorisant l'accès au serveur (VLAN 40) et à Internet.

Listing 17 – ACL.sh — règles iptables de segmentation inter-VLAN

```

1 # 1. Allow loopback
2 ip netns exec sw-core iptables -A FORWARD -i lo -j ACCEPT
3
4 # 2. Allow established connections (return traffic)
5 ip netns exec sw-core iptables -A FORWARD \
6     -m state --state ESTABLISHED,RELATED -j ACCEPT
7
8 # 3. Block inter-client VLAN traffic (VLANs 10, 20, 30)
9 VLAN_CLIENTS=("172.17.1.0/24" "172.17.2.0/24" "172.17.3.0/24")
10 for src in "${VLAN_CLIENTS[@]}; do
11     for dst in "${VLAN_CLIENTS[@]}; do
12         if [ "$src" != "$dst" ]; then
13             ip netns exec sw-core iptables -A FORWARD \
14                 -s "$src" -d "$dst" -j DROP
15         fi
16     done

```

```

17 done
18
19 # 4. Default: ACCEPT (VLAN 40 and Internet remain accessible)
20 ip netns exec sw-core iptables -P FORWARD ACCEPT

```

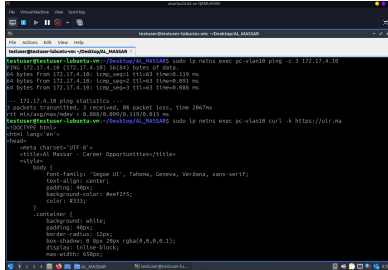


FIGURE 80 — ACL off — ping inter-VLAN autorisé

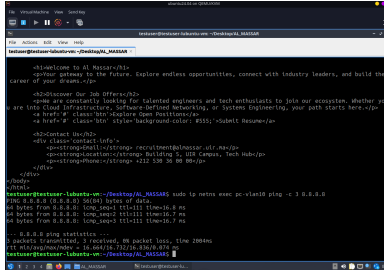


FIGURE 81 — ACL on — ping inter-VLAN bloqué

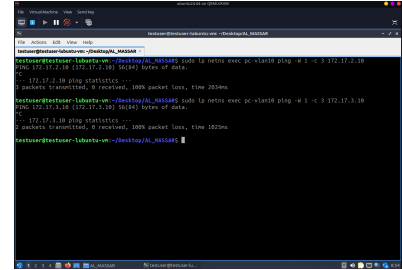


FIGURE 82 — Tentative non autorisée (DROP silencieux)

4.9 Tests et Validation — ALL_TESTS.sh

La suite de tests automatisés valide l'ensemble de la stack réseau depuis le namespace pc-vlan10.

Listing 18 — ALL_TESTS.sh — séquence de tests automatisés

```

1 # [1] DHCP DORA
2 ip netns exec pc-vlan10 dhclient -r veth-pc10 && ip addr flush dev
  veth-pc10
3 ip netns exec pc-vlan10 dhclient -v veth-pc10
4
5 # [2.1] Inter-VLAN routing
6 ip netns exec pc-vlan10 ping -c 2 172.17.3.10
7
8 # [2.2] WAN / NAT
9 ip netns exec pc-vlan10 ping -c 2 8.8.8.8
10
11 # [3.1/3.2] DNS
12 ip netns exec pc-vlan10 nslookup uir.ma
13 ip netns exec pc-vlan10 dig uir.ma
14
15 # [3.3/3.4] HTTPS
16 ip netns exec pc-vlan10 curl -k https://uir.ma
17 ip netns exec pc-vlan10 links https://uir.ma
18
19 # [3.5] FTPS
20 ip netns exec pc-vlan10 lftp -u testuser,123 \
21   -e "set ftp:ssl-force true;set ssl:verify-certificate no;ls;
    quit" \
22   ftps://uir.ma

```

```

testuser@testuser-lubuntu-vm:~/Desktop/AL_MASSAR$ sudo ip netns exec pc-vlan10 ping -c 2 172.17.3.10
[sudo] password for testuser:
PING 172.17.3.10 (172.17.3.10) 56(84) bytes of data:
64 bytes from 172.17.3.10: icmp_seq=1 ttl=63 time=0.141 ms
64 bytes from 172.17.3.10: icmp_seq=2 ttl=63 time=0.059 ms

--- 172.17.3.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1051ms
rtt min/avg/max/mdev = 0.059/0.100/0.141/0.041 ms
testuser@testuser-lubuntu-vm:~/Desktop/AL_MASSAR$ sudo ip netns exec pc-vlan10 ping -c 2 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=111 time=17.0 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=111 time=16.6 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 16.605/16.819/17.033/0.214 ms
testuser@testuser-lubuntu-vm:~/Desktop/AL_MASSAR$ sudo ip netns exec pc-vlan10 ping -c 2 172.17.4.10
PING 172.17.4.10 (172.17.4.10) 56(84) bytes of data:
64 bytes from 172.17.4.10: icmp_seq=1 ttl=63 time=0.043 ms
64 bytes from 172.17.4.10: icmp_seq=2 ttl=63 time=0.061 ms

--- 172.17.4.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1030ms
rtt min/avg/max/mdev = 0.043/0.052/0.061/0.009 ms
testuser@testuser-lubuntu-vm:~/Desktop/AL_MASSAR$

```

FIGURE 83 – Tests de connectivité — inter-VLAN (172.17.3.10) et WAN (8.8.8.8 via NAT)

Les pings valident deux points critiques : le routage inter-VLAN (le paquet depuis 172.17.1.10 atteint 172.17.3.10 en traversant le namespace `sw-core`) et la chaîne NAT complète (le paquet atteint 8.8.8.8 en transitant par `sw-core` → `router` → `host` → WAN). Aucun paquet n'est perdu. Le tableau suivant synthétise l'ensemble des tests effectués sur l'infrastructure virtuelle :

Test	Résultat	Capture
DHCP DORA (pc-vlan10)	OK	Fig. 74
Inter-VLAN routing (pc-vlan10 → pc-vlan30)	OK	Fig. 83
NAT / WAN (ping 8.8.8.8)	OK	Fig. 83
DNS — nslookup uir.ma	OK	Fig. DNS
HTTPS — curl + browser links	OK	Fig. HTTPS
FTPS — lftp TLS forcé	OK	Fig. 79
ACL inter-VLAN (VLAN 10→30)	DROP	Fig. ACL

5 Comparaison des Deux Approches

Critère	Physique	Virtuelle (Namespaces)
Matériel requis	Cisco 2900, Catalyst 3650, 2× HP 2920, serveur Ubuntu physique	Aucun (une seule machine Linux)
Couche L2	Switches réels, VLANs 802.1Q natifs	Bridges Linux (<code>br-sw1/2</code>) + VLAN filtering
Routage inter-VLAN	SVI Catalyst 3650 (<code>ip routing</code>)	SVI bridge dans namespace <code>sw-core</code>
NAT	Cisco IOS (<code>ip nat inside source ... overload</code>)	iptables MASQUERADE sur l'hôte
DHCP	<code>isc-dhcp-server</code> + <code>ip helper-address</code> Cisco	<code>isc-dhcp-server</code> + <code>dhcrelay</code> Linux
DNS	BIND9, zone <code>almassar.local</code>	BIND9, zone <code>uir.ma</code>
HTTPS	Nginx	Apache2
FTPS	vsftpd + TLS	vsftpd + TLS (même config)
Reproductibilité	Dépend du matériel dispo	<code>sudo ./BOOT.sh</code> en 30 secondes
Complexité	Configuration manuelle équipement par équipement	Scripts Bash orchestrés, entièrement automatisé

Conclusion

Ce projet « Al Massar » a permis de déployer une infrastructure réseau d'entreprise complète selon deux approches complémentaires.

La **partie physique** a démontré la rigueur nécessaire à la manipulation d'équipements réels (Cisco IOS, HP ProCurve) : chaque commande doit être précise, l'ordre de configuration importe, et les erreurs sont moins faciles à corriger qu'en environnement logiciel.

La **partie virtuelle** sous Linux Namespaces a mis en évidence la puissance du noyau Linux comme moteur réseau : bridges VLAN-aware, routage entre namespaces, NAT iptables, et services applicatifs complets (DHCP/DNS/HTTPS/FTPS) — le tout déployable en une seule commande (`sudo ./BOOT.sh`) sur n'importe quelle machine Linux. La TUI whiptail ajoute une expérience utilisateur professionnelle et un mode de sécurité ACL configurable à la demande.

L'apport fondamental de ce double déploiement est de montrer que les mêmes concepts réseau (segmentation VLAN, routage inter-VLAN, NAT, services applicatifs) se retrouvent identiques aux deux niveaux, qu'il s'agisse de silicium ou de primitives kernel : **le réseau est une abstraction, et Linux en est une implémentation de référence.**

Références

- **Sujet** : M. Bakhouya, M. Noufel, M. Aqachtoul, M. ElAmrani & M. Staifi, *Administration Réseau — Projet Al Massar*, UIR ESIN, 2025-2026
- **Cours** : M. Bakhouya, M. Noufel, M. Aqachtoul, M. ElAmrani & M. Staifi, *Administration Réseau — Cours et TP*s, UIR ESIN, 2025-2026
- **Documentation Cisco** : Cisco Systems, *Cisco IOS IP Routing Configuration Guide* et *Catalyst 3650 Software Configuration Guide*
- **Documentation HP** : Hewlett-Packard, *HP 2920 Switch Series Management and Configuration Guide*
- **Linux Namespaces** : Michael Kerrisk, *Linux man-pages* — *ip-netns(8)*, *bridge(8)*, *iptables(8)*, kernel.org
- **Cours** : Prof. Pascal SICARD, *Introduction aux Réseaux*, S6, Polytech Grenoble INP, 2024-2025
- **Assistant IA** : Claude (Sonnet 4.6), Anthropic, <https://claude.ai>